Correctness is clear: if $c = g^{q(r)}$ and $y_i = g^{w(r_i)}$ for $i = 1, \ldots, \ell$, then

$$e(c \cdot g^{-v}, g) = e(g^{q(r)-v}, g) = e(g^{\sum_{i=1}^{\ell} w_i(r)\cdot(r_i - z_i)}, g) = \prod_{i=1}^{\ell} e(g^{w_i(r)}, g^{r_i - z_i}) = \prod_{i=1}^{\ell} e(y_i, g^{r_i} \cdot g^{-z_i}).$$

Here, the first inequality holds because $c = g^{q(r)}$, the second holds by Equation (15.4), the third holds by bilinearity of e, and the fourth holds because $y_i = g^{w_i(r)}$.

The proof of binding and techniques to achieve extractability are similar to the previous section and we omit them for brevity.

Costs. Like the univariate pairing-based polynomial commitment scheme of the previous section (Section 15.2), the ℓ-variate multilinear polynomial commitment consists of a constant number of group elements. However, whereas evaluation proofs for the univariate protocol also consisted of a constant number of group elements, evaluation proofs for the multilinear polynomial protocol are ℓ group elements rather than $O(1)$, with a corresponding increase in verification time from $O(1)$ group operations and bilinear map evaluations, to $O(\ell)$.

In terms of committer runtime, Zhang et al. [ZGK$^+$18, Appendix G] show that the committer in the protocol for multilinear polynomials can compute the polynomials $w_1, \ldots, w_\ell$ with just $O(2^\ell)$ field operations in total. And once these polynomials are computed, the prover can compute all ℓ necessary values $g^{w_i(r)}$ with $O(2^\ell)$ many group exponentiations in total.

15.4 Dory: Transparent Scheme with Logarithmic Verification Costs

This section describes a polynomial commitment scheme called Dory with similar (logarithmic) verification costs to the polynomial commitment scheme of the previous section, but that does not rely on a trusted setup. It does require a pre-processing phase that requires time square root in the size of the polynomial to be committed, but this pre-processing phase does not produce any "toxic waste" that must be discarded to guarantee no one can break the scheme's binding property. Asymptotically, its verifier costs compare favorably to Bulletproofs (Section 14.4): like Bulletproofs, it is transparent with logarithmic-sized proofs, but unlike Bulletproofs it has logarithmic rather than linear verifier time. However, concretely its proofs are larger than Bulletproofs by a significant constant factor.

Dory builds on beautiful ideas and building blocks developed over a variety of works [AFG$^+$10, BGH19, BMM$^+$21], especially so-called AFGHO commitments [AFG$^+$10]. While Dory itself is arguably somewhat complicated, the building blocks that comprise it are simpler and useful in their own right.

15.4.1 Commitments to Vectors of Group Elements via Inner Pairing Products

Let $\mathbb{F}_p$ be the field of prime order p and $\mathbb{G}$ be a multiplicative cyclic group of order p. Let $\mathbf{h} = (h_1, \ldots, h_n)$ be a public vector of (randomly chosen) generators of $\mathbb{G}$. Recall that an (unblinded) Pedersen vector commitment is a compressing commitment to a vector of field elements $v \in \mathbb{F}_p^n$, given by $\mathsf{Com}(v) = \prod_{i=1}^n h_i^{v_i}$ (see Section 14.2). In other words, this commitment takes (unblinded) Pedersen commitments to each entry of v, and multiplies them together to get a single commitment to the whole vector. The commitment is a single element of the group $\mathbb{G}$.

Now let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t$ be a pairing-friendly triple of groups of order p. In the previous paragraph, we expressed $\mathbb{G}$ as a multiplicative group, but for the remainder of this section, we will express $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t$ as

additive groups. As in Section 14.4 where we described Bulletproofs, this allows us to express commitments as an inner product between the vector to be committed and the commitment key.

Using pairings, one can define an analog of Pedersen vector commitments that allows one to commit to vectors of *group elements*, i.e., vectors in $\mathbb{G}_1^n$, rather than vectors of field elements. Specifically, for $w \in \mathbb{G}_1^n$, and for a fixed vector $\mathbf{g} = (g_1, \ldots, g_n) \in \mathbb{G}_2^n$ of public, randomly chosen group elements, define

$$\text{IPPCom}(w) = \sum_{i=1}^{n} e(w_i, g_i). \tag{15.5}$$

Note that IPPCom(w) is a single element of the target group $\mathbb{G}_t$. We use the notation IPPCom as short-hand for the term *inner-pairing-product commitment*. This refers to the fact that IPPCom(w) can be thought of as the inner product $\langle w, \mathbf{g} \rangle = \sum_{i=1}^{n} w_i \cdot g_i$, where the "multiplication" of w_i and g_i is defined via the pairing $e(w_i, g_i)$. From now on, for $w \in \mathbb{G}_1^n, \mathbf{g} \in \mathbb{G}_2^n$, we use $\langle w, \mathbf{g} \rangle$ to denote the inner-pairing-product $\sum_{i=1}^{n} e(w_i, g_i)$.

Intuitively, $e(w_i, g_i)$ acts as a Pedersen commitment to w_i despite the fact that w_i is a group element in $\mathbb{G}_1$ rather than a field element in $\mathbb{F}_p$. The sum $\sum_{i=1}^{n} e(w_i, g_i)$ of all such entry-wise Pedersen commitments is the natural compressing commitment to the vector w.

The above commitment scheme for vectors of group elements originated in work of Abe et al. [AFG+10, Gro09] and are often called AFGHO-commitments. We use the terms AFGHO-commitments and inner-pairing-product commitments interchangeably.

Rendering the commitment perfectly hiding. Recall that a Pedersen vector commitment in group $\mathbb{G}_1$ can be made perfectly hiding by having an extra randomly chosen public parameter $g \in \mathbb{G}_1$, and having the committer pick a random $r \in \mathbb{F}_p$ and include a blinding factor g^r in the commitment. Analogously, IPPCom(w) can be made perfectly hiding by having the committer pick a random $r \in \mathbb{G}_1$ and including a blinding term $e(r, g)$ in the commitment, i.e., defining

$$\text{IPPCom}(w) = e(r, g) + \sum_{i=1}^{n} e(w_i, g_i).$$

For simplicity, we omit this blinding factor from the remainder of our treatment.

Computational Binding. Recall from Section 15.1 that the Decisional Diffie-Hellman (DDH) assumption for an additive group $\mathbb{G}_1$ states that no efficient algorithm can meaningfully distinguish between a random tuple of the form

$$(g, a \cdot g, b \cdot g, c \cdot g)$$

with a, b, c chosen at random from $\mathbb{F}_p$ versus one of the form

$$(g, a \cdot g, b \cdot g, (ab) \cdot g).$$

We show that assuming DDH holds in $\mathbb{G}_1$, IPPCom(w) is a computationally binding commitment to $w \in \mathbb{G}_1^n$. For expository clarity, our presentation assumes that $n = 2$.

Given a DDH challenge $(g, a \cdot g, b \cdot g, c \cdot g)$ in $\mathbb{G}_1$, we must explain how to use an efficient prover $\mathcal{P}$ that breaks binding of the commitment scheme to give an efficient algorithm $\mathcal{A}$ that breaks the DDH assumption. $\mathcal{A}$ sets $\mathbf{g} = (g, a \cdot g) \in \mathbb{G}_1 \times \mathbb{G}_1$; note that by definition of the DDH challenge distributions, both entries of $\mathbf{g}$ are uniform random group elements. $\mathcal{A}$ then runs $\mathcal{P}$ to identify a nonzero commitment $c^* \in \mathbb{G}_t$ and two openings $u = (u_1, u_2), w = (w_1, w_2)$ of c^*, meaning that

$$c^* = e(u_1, g) + e(u_2, a \cdot g) = e(w_1, g) + e(w_2, a \cdot g).$$

Let $v = u - w$, and write $v = (v_1, v_2) \in \mathbb{G}_1 \times \mathbb{G}_1$. Since $c^* \neq 0$, it follows that v is not the zero-vector. Moreover, by bilinearity of e,

$$e(v_1, g) + e(v_2, a \cdot g) = 0. \tag{15.6}$$

The DDH adversary $\mathcal{A}$ then outputs 1 if and only if

$$e(v_1, b \cdot g) + e(v_2, c \cdot g) = 0. \tag{15.7}$$

That $\mathcal{A}$ breaks the DDH assumption in $\mathbb{G}_1$ can be seen as follows. First, if $c = a \cdot b$, then the left hand side of Equation (15.7) equals:

$$e(v_1, b \cdot g) + e(v_2, (a \cdot b) \cdot g) = e(v_1, b \cdot g) + e(v_2, b \cdot (a \cdot g)) = b \cdot (e(v_1, g) + e(v_2, a \cdot g)).$$

This last expression equals 0 by Equation (15.6). Hence, in this case $\mathcal{A}$ outputs 1. Meanwhile, if c is chosen at random from $\mathbb{F}_p$, then since v is not the zero-vector, Equation (15.7) holds with probability just $1/p$.

Hence, $\mathcal{A}$ succeeds in distinguishing tuples of the form $(g, a \cdot g, b \cdot g, c \cdot g)$, with a, b, c chosen at random from $\mathbb{F}_p$, from those of the form $(g, a \cdot g, b \cdot g, (ab) \cdot g)$.

15.4.2 Committing to field elements using pairings

One can also use inner-pairing-product commitments in place of Pedersen-vector commitments to commit to vectors of field elements. Let h be any element of $\mathbb{G}_1$ and $\mathbf{g} = (g_1, \ldots, g_n) \in \mathbb{G}_2^n$ be a random vector of $\mathbb{G}_2$ elements. For a vector $v \in \mathbb{F}_p^n$, consider the vector $w(v)$ of entry-wise (unblinded) Pedersen commitments to v in $\mathbb{G}_1$ with commitment key h, i.e., $w(v)_i = v_i \cdot h$, and define IPPCom(v) as

$$\text{IPPCom}(v) = \text{IPPCom}(w(v)) = \langle w(v), \mathbf{g} \rangle = \sum_{i=1}^{n} e(v_i \cdot h, g_i). \tag{15.8}$$

Since IPPCom$(w(v))$ is a binding commitment to $w(v)$, and the map $v \mapsto w(v)$ is bijective, IPPCom(v) is a binding commitment to $v \in \mathbb{F}_p^n$. It will be important both for concrete efficiency of computing commitments, and for the polynomial commitment scheme that we develop (Sections 15.4.4 and 15.4.5), that the *same* group element h is used to compute every entry of $w(v)$.

Efficiency comparison. There are two natural ways to compute IPPCom(v). One is by computing the right hand side of Expression (15.8) directly, which requires n evaluations of the bilinear map e and n group operations in the target group $\mathbb{G}_t$. The other (faster) way is by computing a Pedersen-vector commitment $c = \sum_{i=1}^{n} v_i g_i$ in $\mathbb{G}_2$, and then applying the bilinear map just once to compute $e(h, c)$. By bilinearity of e,

$$e(h, c) = e(h, \sum_{i=1}^{n} v_i \cdot g_i) = \sum_{i=1}^{n} e(h, v_i \cdot g_i) = \sum_{i=1}^{n} e(v_i \cdot h, g_i) = \text{IPPCom}(v).$$

Both methods of computing IPPCom(v) are concretely slower than computing a Pedersen vector commitment to v in $\mathbb{G}_1$. This is because group operations in $\mathbb{G}_2$ and $\mathbb{G}_t$ are typically slower than group operations in $\mathbb{G}_1$, and computing an evaluation of the bilinear map e is slower still. For example, according to microbenchmarks in [Lee21], if using the popular pairing-friendly curve BLS12-381, a $\mathbb{G}_t$ operation is about 4 times slower than one in $\mathbb{G}_1$ while a $\mathbb{G}_2$ operation is about 2 times slower than one in $\mathbb{G}_1$. Moreover,

operations in a pairing-friendly group $\mathbb{G}_1$ such as BLS12-381 are perhaps 2-3 times slower than operations in the fastest non-pairing-friendly groups.

On top of this slow commitment computation, the commitment size IPPCom(v) is concretely larger—for BLS12-381, representations of elements of the target group $\mathbb{G}_t$ are four times bigger than those of elements of $\mathbb{G}_1$. In summary, while the asymptotic costs of computing and sending IPPCom(w) are similar to Pedersen vector commitments, the concrete costs are worse.

A simplification in the remainder of the presentation. As discussed in Section 15.1, the DDH assumption *cannot* hold in $\mathbb{G}_1$ if $\mathbb{G}_1 = \mathbb{G}_2$, i.e., if the pairing is a symmetric pairing. This is because, given tuple (g, ag, bg, cg), one can check whether $c = a \cdot b$ by checking whether $e(g, c \cdot g) = e(a \cdot g, b \cdot g)$.

However, the DDH assumption can hold in $\mathbb{G}_1$ and $\mathbb{G}_2$ if the two groups are not equal, i.e., if there is no efficiently computable mapping ϕ group $\mathbb{G}_1$ to $\mathbb{G}_2$ or vice versa that preserves group structure in the sense that $\phi(a+b) = \phi(a) + \phi(b)$. This is (believed to be) the case for pairings used in practice, such as BLS12-381. The assumption that DDH holds in both $\mathbb{G}_1$ and $\mathbb{G}_2$ is called the *symmetric external Diffie-Hellman assumption*, abbreviated SXDH.

Despite the fact that IPPCom(w) is *not* necessarily a binding commitment to w if $\mathbb{G}_1 = \mathbb{G}_2$, we will nonetheless assume for the remainder of our presentation of Dory that $\mathbb{G}_1 = \mathbb{G}_2$, denoting both groups by $\mathbb{G}$.[186] This simplifies the presentation of the Dory protocol. The changes required to the protocol in the case where $\mathbb{G}_1 \neq \mathbb{G}_2$ are straightforward, but they introduce a notational burden that we prefer to avoid.

Outline for the remainder of the presentation. In order to highlight in a modular fashion the main ideas that go into the Dory polynomial commitment scheme, we describe progressively more sophisticated protocols. First, in Section 15.4.3, we explain how a prover $\mathcal{P}$ can convince a verifier that $\mathcal{P}$ knows how to open some inner-pairing-product commitment $c_u = \text{IPPCom}(u)$ to some vector $u \in \mathbb{G}^n$. The proof size and verification cost of this protocol are $O(\log^2 n)$, after a transparent linear-time pre-processing phase. Second, in Section 15.4.4 we explain how to extend this protocol to give a transparent polynomial commitment scheme in which the verifier's runtime is $O(\log^2 n)$. Third, in Section 15.4.5, we explain a modification of the protocol of Section 15.4.3 that reduces the runtime of the pre-processing phase from linear in n to $O(n^{1/2})$. Finally, in Section 15.4.6, we present a more complicated variant of the protocol of Section 15.4.3 that reduces verification costs from $O(\log^2 n)$ to $O(\log n)$.

15.4.3 Proving Knowledge of Opening with $O(\log^2 n)$ Verification Cost

Let $u \in \mathbb{G}^n$, and assume for simplicity that n is a power of 2. Given public input $c_u = \text{IPPCom}(u)$, the following protocol is transparent and allows the prover to prove knowledge of an opening u of c_u. The verifier runs in $O(\log^2 n)$ time, after a transparent pre-processing phase that is independent of u.

Recap of Bulletproofs. Let us briefly recall the Bulletproofs (Section 14.4) protocol for establishing knowledge of an opening $u^{(0)} \in \mathbb{F}_p^n$ of a *Pedersen* vector commitment

$$c_{u^{(0)}} = \langle u^{(0)}, \mathbf{g}^{(0)} \rangle = \sum_{i=1}^{n} u_i \cdot g_i.$$

[186] See [Gro09] for a modification of the commitment scheme that is binding under a plausible assumption for symmetric pairings.

Conceptually, $\mathbf{g}^{(0)}$ is broken into two halves $\mathbf{g}_L^{(0)}$ and $\mathbf{g}_R^{(0)}$, and likewise $u^{(0)}$ is written as $(u_L^{(0)}, u_R^{(0)})$. The prover sends two commitments v_L and v_R claimed to equal $\langle u_L^{(0)}, \mathbf{g}_R^{(0)} \rangle$ and $\langle u_R^{(0)}, \mathbf{g}_L^{(0)} \rangle$. The verifier picks a random $\alpha_1 \in \mathbb{F}_p$ and sends it to $\mathcal{P}$, and uses v_L and v_R to homomorphically update the commitment $c_{u^{(0)}}$ to

$$c_{u^{(1)}} := c_{u^{(0)}} + \alpha_1^2 v_L + \alpha_1^{-2} v_R.$$

If the prover is honest, $c_{u^{(1)}}$ is a commitment to the to length-$(n/2)$ vector

$$u^{(1)} = \alpha_1 u_L^{(0)} + \alpha_1^{-1} u_R^{(0)}.$$

using the commitment key

$$\mathbf{g}^{(1)} := \alpha_1^{-1} \mathbf{g}_L^{(0)} + \alpha_1 \mathbf{g}_R^{(0)}. \tag{15.9}$$

$\mathcal{P}$ and $\mathcal{V}$ then proceed to the next round, in which $\mathcal{P}$ recursively establishes knowledge of an opening $u^{(1)}$ to $c_{u^{(1)}}$. This continues for $\log n$ rounds, at which point the recursion bottoms out: $u^{(\log n)}$ and $\mathbf{g}^{(\log n)}$ have length 1, and hence (if zero-knowledge is not a consideration) $\mathcal{P}$ can afford to prove knowledge of $u^{(\log n)}$ by sending it explicitly to $\mathcal{V}$, who can check that $u^{(\log n)} \cdot \mathbf{g}^{(\log n)} = c_{u^{(\log n)}}$.

Why is the verifier runtime in the above protocol linear rather than logarithmic? The answer is: in each round, the verifier needs to update the commitment key from $\mathbf{g}^{(i-1)}$ to $\mathbf{g}^{(i)} = \alpha_i^{-1} \mathbf{g}_L^{(i-1)} + \alpha \mathbf{g}_R^{(i-1)}$. This takes time at least $O(n/2^i)$.

The pre-processing procedure. Now let $u^{(0)}$ be a vector in $\mathbb{G}^n$, and let $c_{u^{(0)}} = \text{IPPCom}(u^{(0)})$. To avoid linear verifier time in our protocol for establishing knowledge of an opening $u^{(0)} \in \mathbb{G}^n$ for $c_{u^{(0)}}$, we rely on a pre-processing phase that is independent of $u^{(0)}$, depending only on the public commitment key $\mathbf{g}$. For exposition, we describe the preprocessing as occurring over $\log n$ iterations, with two inner-pairing-product commitments (i.e., elements of $\mathbb{G}_t$) produced per iteration. We refer to the party doing the pre-processing as the verifier—in fact, any entity willing to invest the effort can perform the pre-processing and distribute the resulting (logarithmically-many) commitments to the world. Any entity willing to invest the effort can also validate the distributed commitments, raising an alarm if a discrepancy is found.

- (First iteration of pre-processing): Let $\mathbf{g}^{(0)} = (\mathbf{g}_L^{(0)}, \mathbf{g}_R^{(0)}) \in \mathbb{G}^{n/2} \times \mathbb{G}^{n/2}$ be the commitment key used to compute the initial commitment $c_{u^{(0)}} = \langle u^{(0)}, \mathbf{g}^{(0)} \rangle$. The first pre-processing iteration outputs inner-pairing-product commitments $\Delta_L^{(1)}$ and $\Delta_R^{(1)}$ to $\mathbf{g}_L^{(0)}$ and to $\mathbf{g}_R^{(0)}$ respectively, using public, randomly chosen commitment key $\Gamma^{(1)} \in \mathbb{G}^{n/2}$.[187] That is, $\Delta_L^{(1)} = \langle \mathbf{g}_L^{(0)}, \Gamma^{(1)} \rangle$ and $\Delta_R^{(1)} = \langle \mathbf{g}_R^{(0)}, \Gamma^{(1)} \rangle$.

- (Second iteration): Write $\Gamma^{(1)}$ itself as $(\Gamma_L^{(1)}, \Gamma_R^{(1)}) \in \mathbb{G}^{n/4} \times \mathbb{G}^{n/4}$. The second iteration computes commitments $\Delta_L^{(2)}$ and $\Delta_R^{(2)}$ to $\Gamma_L^{(1)}$ and to $\Gamma_R^{(1)}$ respectively, using public, randomly chosen commitment key $\Gamma^{(2)} \in \mathbb{G}^{n/4}$.[188]

- In general, in iteration $i > 1$: compute commitments $\Delta_L^{(i)}$ and $\Delta_R^{(i)}$ to the left and right halves of the commitment key $\Gamma^{(i-1)}$ that was used to compute the previous iteration's commitments $\Delta_L^{(i-1)}$ and $\Delta_R^{(i-1)}$.

[187] $\Gamma^{(1)}$ need not be independent of $\mathbf{g}^{(0)}$, e.g., it is fine for $\Gamma^{(0)}$ to equal $\mathbf{g}_L^{(0)}$.

[188] As in Footnote 187, $\Gamma^{(2)}$ need not be independent of $\Gamma^{(1)}$, e.g., it is fine for $\Gamma^{(2)}$ to equal $\Gamma_L^{(1)}$.

The pre-processing ends after iteration $i = \log(n)$. At that point, $\Gamma^{(i)}$ has length 1, so the pre-processing just outputs $\Gamma^{(i)}$ explicitly. Note that after the pre-processing is done, a logarithmic-time verifier does have time to read and store the two commitments $\Delta_L^{(i)}$ and $\Delta_R^{(i)}$ output by each iteration of pre-processing, but does *not* have the time to read or store the corresponding *commitment key* $\Gamma^{(i)}$ used to produce $\Delta_L^{(i)}$ and $\Delta_R^{(i)}$. This is because $\Gamma^{(i)}$ has size $n/2^i$ and hence is super-logarithmic for all $i \leq \log(n) - \log\log(n)$. As we will see, this means the verifier in the knowledge-of-opening protocol described below will have to somehow "check" that the prover knows how to open many different commitments $\Delta_L^{(i)}$ and $\Delta_R^{(i)}$ *without the verifier even knowing the keys* used to produce those commitments.

The knowledge of opening protocol. Let $u^{(0)}$ be a vector in $\mathbb{G}^n$. The verifier begins the protocol knowing a commitment $c_{u^{(0)}}$. If the prover is honest,

$$c_{u^{(0)}} = \langle u^{(0)}, \mathbf{g}^{(0)} \rangle, \tag{15.10}$$

and the prover needs to prove that it knows a vector $u^{(0)}$ satisfying Equation (15.10). Recall that a core difficulty here is that the verifier doesn't know $\mathbf{g}^{(0)}$ as in Bulletproofs, but rather only some "pre-processed" information about $\mathbf{g}^{(0)}$, namely commitments to $\mathbf{g}_L^{(0)}$ and $\mathbf{g}_R^{(0)}$ under some different commitment key $\Gamma^{(1)}$.

The key idea is that, in each round i, rather than explicitly computing $\mathbf{g}^{(i)}$ from $\mathbf{g}^{(i-1)}$ as per Equation (15.9), $\mathcal{V}$ instead uses homomorphism of the commitments output by the pre-processing procedure to compute in constant time a *commitment* to $\mathbf{g}^{(i)}$ under a suitable commitment key. Roughly speaking, the verifier can use this commitment to force the prover to do the hard work of computing $\mathbf{g}^{(i)}$ explicitly. The prover will only ever explicitly reveal to the verifier the final "fully collapsed" commitment key $\mathbf{g}^{(\log n)}$, which consists of a single group element. Protocol details follow.

Round 1 **procedure**: As in Bulletproofs, the prover begins by sending two commitments v_L and v_R claimed to equal $\langle u_L^{(0)}, \mathbf{g}_R^{(0)} \rangle$ and $\langle u_R^{(0)}, \mathbf{g}_L^{(0)} \rangle$. The verifier picks a random $\alpha_1 \in \mathbb{F}_p$ and sends it to $\mathcal{P}$. The verifier uses v_L and v_R to homomorphically update the commitment $c_{u^{(0)}}$ to

$$c_{u^{(1)}} := c_{u^{(0)}} + \alpha_1^2 v_L + \alpha_1^{-2} v_R.$$

Recall that unlike in Bulletproofs, our $\mathcal{V}$ does not know $\mathbf{g}^{(0)}$, but does know pre-processing commitments $\Delta_L^{(1)} = \langle g_L^{(0)}, \Gamma^{(1)} \rangle$ and $\Delta_R^{(1)} = \langle g_R^{(0)}, \Gamma^{(1)} \rangle$. Via homomorphism, $\mathcal{V}$ can compute a commitment

$$c_{\mathbf{g}^{(1)}} = \alpha_1^{-1} \Delta_L^{(1)} + \alpha_1 \Delta_R^{(1)}$$

to $\mathbf{g}^{(1)} = \alpha_1^{-1} \mathbf{g}_L^{(0)} + \alpha_1 \mathbf{g}_R^{(0)}$ under commitment key $\Gamma^{(1)}$.

The above Round 1 leaves the prover and verifier in the following situation. Unlike in Bulletproofs, at the start of Round 2, our verifier does not know $\mathbf{g}^{(1)}$. All that our verifier knows is a *commitment* $c_{\mathbf{g}^{(1)}}$ to $\mathbf{g}^{(1)}$ under commitment key $\Gamma^{(1)} = (\Gamma_L^{(1)}, \Gamma_R^{(1)}) \in \mathbb{G}^{n/4} \times \mathbb{G}^{n/4}$.

Because the information known to $\mathcal{V}$ is so limited, at the start of Round 2, $\mathcal{P}$ needs to prove that it knows vectors $u^{(1)}$ and $\mathbf{g}^{(1)}$ in $\mathbb{G}^{n/2}$ such that

$$c_{u^{(1)}} = \langle u^{(1)}, \mathbf{g}^{(1)} \rangle \tag{15.11}$$

and

$$c_{\mathbf{g}^{(1)}} = \langle \mathbf{g}^{(1)}, \Gamma^{(1)} \rangle. \tag{15.12}$$

In summary, the first round of our protocol started with a claim about *one* inner product equation involving two vectors $u^{(0)}$ and $\mathbf{g}^{(0)}$ of length n (Equation (15.10)), in which $\mathcal{V}$ only knew "pre-computed commitments" $\Delta_L^{(1)}, \Delta_R^{(1)}$ to $\mathbf{g}^{(0)}$. And it reduced it to a claim about *two* inner product equations involving *three* vectors of length $n/2$, namely $u^{(1)}$, $\mathbf{g}^{(1)}$, and $\Gamma^{(1)}$, in which $\mathcal{V}$ only knows pre-computed commitments $\Delta_L^{(2)}, \Delta_R^{(2)}$ to $\Gamma_L^{(1)}$ and $\Gamma_R^{(1)}$.

Round 2 procedure.

- *A trivial case:* If $n = 2$, so the pre-processing phase output $\Gamma^{(1)}$ explicitly, then a trivial way for the prover to establish both of these claims is to explicitly reveal $u^{(1)}$ and $\mathbf{g}^{(1)}$ to $\mathcal{V}$, who can then check Equations (15.11) and (15.12) explicitly. But this does not work if $n \geq 4$.

- *What to do if $n \geq 4$.* Fortunately, Equations (15.11) and (15.12) are both of a form that Bulletproofs was designed to handle. Namely, $\mathcal{P}$ is claiming to know some vector satisfying an inner-product relation with some other vector ($u^{(1)}$ with $\mathbf{g}^{(1)}$ in Equation (15.11) and $\mathbf{g}^{(1)}$ with $\Gamma^{(1)}$ in Equation (15.12)). So $\mathcal{P}$ and $\mathcal{V}$ can apply the Bulletproofs scheme in parallel to both claims, using the same random verifier-chosen $\alpha_2 \in \mathbb{F}_p$ for both, to reduce each claim to an equivalent one involving vectors of half the length.

  That is, $\mathcal{P}$ sends commitments $v_L, v_R, w_L, w_R \in \mathbb{G}_t$ claimed to equal $\langle u_L^{(1)}, \mathbf{g}_R^{(1)} \rangle$, $\langle u_R^{(1)}, \mathbf{g}_L^{(1)} \rangle$, $\langle \mathbf{g}_L^{(1)}, \Gamma_R^{(1)} \rangle$ and $\langle \mathbf{g}_R^{(1)}, \Gamma_L^{(1)} \rangle$. $\mathcal{V}$ then sends $\alpha_2 \in \mathbb{F}$ to $\mathcal{P}$. $\mathcal{V}$ sets

$$c_{u^{(2)}} \coloneqq c_{u^{(1)}} + \alpha_2^2 v_L + \alpha_2^{-2} v_R.$$

  If $\mathcal{P}$ is honest, then

$$c_{u^{(2)}} = \langle u^{(2)}, \mathbf{g}^{(2)} \rangle,$$

  where

$$u^{(2)} = \alpha_2 u_L^{(1)} + \alpha_2^{-1} u_R^{(1)},$$

  and

$$\mathbf{g}^{(2)} \coloneqq \alpha_2^{-1} \mathbf{g}_L^{(1)} + \alpha_2 \mathbf{g}_R^{(1)}.$$

  Likewise, $\mathcal{V}$ sets

$$c_{\mathbf{g}^{(2)}} \coloneqq c_{\mathbf{g}^{(1)}} + \alpha_2^{-2} w_L + \alpha_2^2 w_R.$$

  If $\mathcal{P}$ is honest, then $c_{\mathbf{g}^{(2)}}$ is a commitment to $\mathbf{g}^{(2)}$ under commitment key $\Gamma' = \alpha_2 \Gamma_L^{(1)} + \alpha_2^{-1} \Gamma_R^{(1)}$, i.e.,

$$c_{\mathbf{g}^{(2)}} = \langle \alpha_2^{-1} \mathbf{g}_L^{(1)} + \alpha_2 \mathbf{g}_R^{(1)}, \alpha_2 \Gamma_L^{(1)} + \alpha_2^{-1} \Gamma_R^{(1)} \rangle.$$

  Finally, $\mathcal{V}$ computes a commitment $c_{\Gamma'}$ to Γ' under commitment key $\Gamma^{(2)}$ homomorphically given the pre-processing commitments $\Delta_L^{(2)}$ and $\Delta_R^{(2)}$, via $c_{\Gamma'} = \alpha_2 \Delta_L^{(2)} + \alpha_2^{-1} \Delta_R^{(2)}$.

The above Round 2 procedure reduces the task of proving knowledge of $u^{(1)}, \mathbf{g}^{(1)} \in \mathbb{G}^{n/2}$ satisfying Equations (15.11) and (15.12) to proving knowledge of $u^{(2)}, \mathbf{g}^{(2)}, \Gamma' \in \mathbb{G}^{n/4}$ satisfying:

$$c_{u^{(2)}} = \langle u^{(2)}, \mathbf{g}^{(2)} \rangle, \tag{15.13}$$

$$c_{\mathbf{g}^{(2)}} = \langle \mathbf{g}^{(2)}, \Gamma' \rangle, \tag{15.14}$$

$$c_{\Gamma'} = \langle \Gamma', \Gamma^{(2)} \rangle. \tag{15.15}$$

That is, as discussed in the "sketch of the knowledge extractor" several paragraphs hence, $\mathcal{P}$'s knowledge of $u^{(2)}, g^{(2)}, \Gamma' \in \mathbb{G}^{n/4}$ satisfying Equations (15.13)-(15.15) (for at least 3 values of $\alpha_2 \in \mathbb{F}_p$) implies knowledge of $u^{(1)}$ and $\mathbf{g}^{(1)}$ satisfying Equations (15.11) and (15.12).

In summary, Round 2 started with a claim about *two* inner product equations involving *three* vectors of length $n/2$, namely $u^{(1)}, \mathbf{g}^{(1)}, \Gamma^{(1)}$, in which $\mathcal{V}$ only knows pre-computed commitments $\Delta_L^{(2)}, \Delta_R^{(2)}$ to $\Gamma^{(1)}$. And it reduced it to a claim about *three* inner product equations involving *four* vectors of length $n/4$, in which $\mathcal{V}$ only knows pre-computed commitments to the fourth vector $\Gamma^{(2)}$.

Round $i > 2$. The above Round-2 procedure can be iterated, to ensure that at the start of Round i, the prover has to establish knowledge of $i+1$ vectors, each of length $n/2^i$, satisfying i inner product equations. In more detail, denote the $i+1$ vectors $\mathcal{P}$ claims to know at the start of round i by $v_1, \ldots, v_{i+1}$. Then $v_1 = u^{(i-1)}$, $v_2 = \mathbf{g}^{(i-1)}$, and $v_{i+1} = \Gamma^{(i-1)}$. And the j'th equation at the start of Round i is of the form $\langle v_j, v_{j+1} \rangle = c_j$ for some commitment c_j.

The prover operates analogously to Round 2, sending commitments to two cross-terms per equation, with the verifier then picking a random $\alpha_i \in \mathbb{F}_p^n$ and sending it to $\mathcal{P}$. For each odd $j \in \{1, \ldots i+1\}$, the verifier uses homomorphism to derive a commitment to $\alpha_i \cdot v_{j,L} + \alpha_i^{-1} \cdot v_{j,R}$. Likewise, for each even j, $\mathcal{V}$ derives a commitment to $\alpha_i^{-1} \cdot v_{j,L} + \alpha_i \cdot v_{j,R}$. For $v_{i+1} = \Gamma^{(i-1)}$, the appropriate commitment is derived homomorphically from the pre-processing commitments $\Delta_L^{(i)}$ and $\Delta_R^{(i)}$. Round $i+1$ is then devoted to proving that the prover indeed knows how to open each of the $i+1$ derived commitments, which entails $i+1$ inner-product equations involving $i+2$ vectors.

The iterations stop after round $\log(n)$; at that point, the vectors involved in each of the $\log(n)+1$ equations have length just 1. If zero-knowledge is not a consideration, the prover can establish that it knows such vectors by simply sending them explicitly to $\mathcal{V}$, and $\mathcal{V}$ can directly check that the received values indeed satisfy all the equations claimed.

Verification costs. The total number of commitments sent by the prover in Round i is $O(i)$, leading to a communication cost $O(\sum_{i=1}^{\log(n)} i) = O(\log^2 n)$. The verifier's total runtime is also $O(\log^2 n)$ scalar multiplications in $\mathbb{G}_t$.

Sketch of the knowledge extractor. The knowledge extractor for the above protocol proceeds similarly to that for Bulletproofs (Section 14.4). After first generating a 3-transcript tree via the forking lemma (Theorem 14.1), the extractor proceeds from the leaves toward the root, and at each vertex i layers below the root it constructs the $i+1$ vectors satisfying the i inner-product equations that the prover claims to know at the point in the protocol corresponding to that vertex. For example, when the extractor comes to the root itself, the extractor has already constructed, for each child of the root, vectors $u^{(1)}$ and $\mathbf{g}^{(1)}$ such that Equations (15.11) and (15.12) hold, i.e., $\langle u^{(1)}, \mathbf{g}^{(1)} \rangle = c_{u^{(1)}}$ and $\langle \mathbf{g}^{(1)}, \Gamma^{(1)} \rangle = c_{\mathbf{g}^{(1)}}$. Note the extractor knows $\Gamma^{(1)}$, because $\Gamma^{(1)}$ is public and, unlike the verifier, the extractor need only run in polynomial time, not logarithmic time.

The Bulletproofs extractor (Section 14.4) gives a procedure to take the $\mathbf{g}^{(1)}$ vectors for all three children of the root and reconstruct a $\mathbf{g}^{(0)}$ that "explains" all three children's $\mathbf{g}^{(1)}$ vectors, in the sense that for each child of the root, $\mathbf{g}^{(1)} = \alpha_1^{-1}\mathbf{g}_L^{(0)} + \alpha_1\mathbf{g}_R^{(0)}$, where α_1 is label of the edge connecting the root to the child under consideration. Once $\mathbf{g}^{(0)}$ is identified, the same Bulletproofs extraction procedure identifies a vector $u^{(0)}$ that explains all three children's $u^{(1)}$ vectors, i.e., such that at each child of the root, $u^{(1)} = \alpha_1 u_L^{(0)} + \alpha_1^{-1}u_R^{(0)}$, and moreover $c_{u^{(0)}} = \langle u^{(0)}, \mathbf{g}^{(0)}\rangle$.

15.4.4 Extending to a polynomial commitment

Extending the knowledge-of-opening protocol of Section 15.4.3 to a polynomial commitment scheme follows the outline provided at the start of the chapter: let $a \in \mathbb{F}_p^n$ be the coefficient vector of the polynomial $q(X) = \sum_{i=0}^{n-1} a_i X^i$ to be committed, of degree $n-1$. Then the commitment to q is just an AFGHO-commitment to a (Section 15.4.2).[189]

Recall this means the following. If $\mathbf{g} = (g_1, \ldots, g_n) \in \mathbb{G}^n$ and $h \in \mathbb{G}$ are public commitment keys, and $w(a) \in \mathbb{G}^n = (a_1 \cdot h, \ldots, a_n \cdot h)$ is the vector of Pedersen commitments to a with key $h \in \mathbb{G}$, then the commitment to the polynomial is $c_q := \text{IPPCom}(w(a)) = \prod_{i=1}^n e(a_i \cdot h, g_i)$.

Suppose the verifier then requests to evaluate the committed polynomial at input $r \in \mathbb{F}_p$, and the prover claims that $q(r) = v$. Then the prover has to establish that it knows a vector a such that two equalities hold: (1) $c_q = \text{IPPCom}(w(a))$, and (2) $q(r) = v$. Recall that the latter claim is equivalent to $\langle a, y\rangle = v$ where $y = (1, r, r^2, \ldots, r^{n-1})$.

Claim (1) is established by applying the protocol of Section 15.4.3 establish that $\mathcal{P}$ knows $u := w(a)$ opening c_q. Claim (2) is established in parallel with this protocol, in close analogy to the Bulletproofs polynomial commitment scheme (Protocol 13). In slightly more detail, there are two differences from Protocol 13, aside from using the protocol of Section 15.4.3 in place of Protocol 12 to establish that $\mathcal{P}$ knows an opening of c_q:

- In Protocol 13, the verifier explicitly computes the vector $y^{(i)}$ in each round. This requires time $O(n/2^i)$ in round i, which is far too large for the polylogarithmic time verifier we set out to achieve in this section. To address this, the key observation is that $\mathcal{V}$ does not actually need to know $y^{(i)}$ for $i < \log n$. The only information the verifier actually needs to perform its checks in the protocol is the final-round value $y^{(\log n)}$. Moreover, not only does $y = (1, r, \ldots, r^{n-1})$ have "tensor structure", but so do the round-by-round updates to y. This enables $\mathcal{V}$ to compute $y^{(\log n)}$ in logarithmic time.

  Specifically, for $(a, b) \in \mathbb{F}_p^2$ and a vector $v \in \mathbb{F}_p^n$, define the vector $v \otimes (a, b)$ to be $(a \cdot v, b \cdot v) \in \mathbb{F}_p^{2n}$. Then it can be checked that $y = \otimes_{i=0}^{\log(n)-1}(1, r^{2^i})$. For example,

  $$(1, r) \otimes (1, r^2) = (1, r, r^2, r^3)$$

  and

  $$(1, r) \otimes (1, r^2) \otimes (1, r^4) = (1, r, r^2, \ldots, r^7).$$

  For $i \in \{1, \ldots, \log n\}$, let $\bar{i} = \log(n) - i$. It can be checked that the above tensor structure in the vector y leads to the following equality:

  $$y^{(\log n)} = \prod_{i=1}^{\log n}(\alpha_i + \alpha_i^{-1} r^{2^{\bar{i}}}). \tag{15.16}$$

[189] Recall that the same approach applies to multilinear polynomials; we restrict our attention to univariate polynomials in this section for brevity.

Clearly, the right hand side of Equation (15.16) can be computed in $O(\log n)$ time.

For example, if $n = 4$, then

$$y^{(1)} = (1, r, r^2, r^3),$$

$$y^{(2)} = (\alpha_1 \cdot 1 + \alpha_1^{-1} \cdot r^2, \alpha_1 r + \alpha_1^{-1} r^3),$$

and

$$y^{(3)} = \alpha_2 y_L^{(2)} + \alpha_2^{-1} y_R^{(2)} = \alpha_2 \cdot \alpha_1 \cdot 1 + \alpha_2^{-1} \cdot \alpha_1 \cdot r + \alpha_2 \cdot \alpha_1^{-1} r^2 + \alpha_2^{-1} \alpha_1^{-1} r^3 = \left(\alpha_1 + \alpha_1^{-1} r^2\right)\left(\alpha_2 + \alpha_2^{-1} r\right).$$

- In the final round, round $\log n$, of the protocol of Section 15.4.3, the prover should reveal not only the group element $u^{(\log n)}$ such that $e(u^{(\log n)}, \mathbf{g}^{(\log n)}) = c_{u^{(\log n)}}$, but also the discrete logarithm $a^* \in \mathbb{F}_p$ to base h, i.e., the field element to which $u^{(\log n)}$ is a Pedersen commitment. This enables the verifier to confirm that indeed $\langle a^*, y^{(\log n)} \rangle = a^* \cdot y^{(\log n)} = v^{(\log n)}$, where $y^{(i)}$ and $v^{(i)}$ denote the round-i values of the vectors y and v from Protocol 13.

15.4.5 Reducing Pre-Processing Time to $O(\sqrt{n})$ via Matrix Commitments

Analogy to Hyrax polynomial commitment. Recall that Section 14.2 used Pedersen vector commitments to give a constant-size polynomial commitment, but with linear-size evaluation proofs and verification time. Section 14.3 reduced the evaluation proof size and verification time to square root, but at the cost of increasing commitment size from constant to square root. It worked by expressing any polynomial evaluation query $q(z)$ as a vector-matrix-vector product $b^T \cdot u \cdot a$ and having the prover commit to each column of the matrix u separately. The verifier could then use homomorphism of the column commitments to compute a commitment to $u \cdot a$, and the prover could then invoke the evaluation procedure of the commitment scheme to reveal $b^T \cdot (u \cdot a)$.

Unlike the commitment scheme of Section 14.2, the schemes of this section *already* have (poly)logarithmic verification costs, following a linear-time pre-processing phase. It is nonetheless possible to combine this section's commitment schemes with the vector-matrix-vector multiplication structure in evaluation queries, to improve other costs. Specifically, the pre-processing time can be reduced from linear to square root in the degree of the committed polynomial. And unlike in Section 14.3, this improvement does not come at the cost of increasing the size of the commitment. This technique has the added benefit that polynomial evaluation queries can be answered with $O(n^{1/2})$ rather than $O(n)$ group operations by the prover (on top of the $O(n)$ $\mathbb{F}_p$ operations required to simply evaluate the polynomial at the requested point).

Commitment phase of the improved protocol. Let $u \in \mathbb{F}_p^{m \times m}$ be the matrix such that for any $z \in \mathbb{F}_p$, $q(z) = b^T \cdot u \cdot a$ for some vectors $b, a \in \mathbb{F}_p^m$. The prover "in its own head" computes a Pedersen-vector commitment c_i in $\mathbb{G}$ to each column i of u, using random generator vector $\mathbf{h} = (h_1, \ldots, h_m) \in \mathbb{G}^m$ as the commitment key. Rather than sending $(c_1, \ldots, c_m) \in \mathbb{G}^m$ explicitly to the verifier, the prover instead just sends an inner-pairing-product commitment c^* to $(c_1, \ldots, c_m)$ using $\mathbf{g} = (g_1, \ldots, g_m)$ as the commitment key. In other words, the commitment is

$$c^* := \sum_{j=1}^{m} e\left(\sum_{i=1}^{m} u_{i,j} \cdot h_i, g_j\right) = \sum_{j=1}^{m} \sum_{i=1}^{m} e\left(u_{i,j} \cdot h_i, g_j\right).$$

This is just a single element of $\mathbb{G}_t$.

Evaluation phase of the improved protocol. If the verifier requests the evaluation $q(z)$, let $w,x \in \mathbb{F}_p^m$ be vectors such that $q(z) = w^T \cdot u \cdot x$, and let v denote the claimed value of $q(z)$.

Both vectors w and x themselves have the same tensor structure exploited to maintain polylogarithmic verifier time in Section 15.4.4. Recall that that protocol allowed a prover to establish knowledge of an opening $\mathbf{t} \in \mathbb{G}^m$ of a given inner-pairing-product commitment such that $\langle \mathbf{t}, y \rangle = v$, where $y \in \mathbb{F}^m$ is any public vector that can be written as a $(\log m)$-dimensional tensor product of length-2 vectors.

Denote $u \cdot x \in \mathbb{F}_p^m$ by $\mathbf{d}$. The prover first sends the verifier a Pedersen-vector commitment $c' \in \mathbb{G}$, whose prescribed value is $\langle \mathbf{d}, \mathbf{h} \rangle$. It suffices for the prover to establish three things.

- The prover knows an opening $\mathbf{t} = (t_1, \ldots, t_m) \in \mathbb{G}^m$ of the inner-pairing-product commitment c^*.
- $\langle \mathbf{t}, x \rangle = c'$. Combined with the first bullet above, this means that, if u is the matrix with column commitments given by the entries of $\mathbf{t}$, then c' is a Pedersen-vector commitment to $u \cdot x$ using key $\mathbf{h}$, i.e., $c' = \sum_{i=1}^m (u \cdot x)_i \cdot h_i$.
- The prover knows an opening $\mathbf{d} = (d_1, \ldots, d_m) \in \mathbb{F}_p^m$ of c' such that $\langle w, \mathbf{d} \rangle = v$. Combined with the bullet above, this means that $w^T \cdot u \cdot x = v$ as claimed by $\mathcal{P}$.

The first two items together can be established directly by the protocol of Section 15.4.4.

Since w also has tensor structure, it is tempting to also apply the protocol of Section 15.4.4 to establish that the third bullet point holds. This does not work directly because c' is not an AFGHO-commitment to $\mathbf{d}$ but rather a Pedersen-vector commitment to $\mathbf{d}$. To address this, the verifier simply transforms c' into an AFGHO-commitment as follows: for a public, randomly chosen $g \in \mathbb{G}$, the verifier lets $c'' = e(c', g)$. Then $c'' \in \mathbb{G}_t$ is an AFGHO-commitment to $\mathbf{d}$ with commitment keys $\mathbf{h} \in \mathbb{G}^n$ and $g \in \mathbb{G}$. Indeed, by bilinearity of e:

$$c'' = e(\sum_{i=1}^m d_i h_i, g) = \sum_{i=1}^n e(d_i \cdot h_i, g) = \sum_{i=1}^n e(h_i, d_i \cdot g).$$

Hence, the prover can invoke the protocol of Section 15.4.4 to prove it knows an opening $\mathbf{d}$ of c'' such that $\langle w, \mathbf{d} \rangle = v$.

15.4.6 Achieving $O(\log n)$ communication and verification time

Recall that in Round 1 of the protocol of Section 15.4.3, the verifier is able to use the pre-processing outputs $\Delta_L^{(1)}$ and $\Delta_R^{(1)}$ to compute a commitment $c_{\mathbf{g}^{(1)}}$ to the newly-updated vector $\mathbf{g}^{(1)} = \alpha_1^{-1}\mathbf{g}_L^{(0)} + \alpha_1 \mathbf{g}_R^{(0)}$ under key $\Gamma^{(1)}$. However, the verifier is *not* able to derive a commitment to $\mathbf{u}^{(1)} = \alpha_1 u_L^{(0)} + \alpha_1^{-1} u_R^{(0)}$ under key $\Gamma^{(1)}$. Rectifying this turns out to lead to improved verification costs, of $O(\log n)$ instead of $O(\log^2 n)$.

The protocol below conceptually proceeds in $\log n$ *rounds* in close analogy with Bulletproofs and the protocol of Section 15.4.3, with each round halving the lengths of the vectors under consideration. But each round in the protocol below actually consists of 4 messages. So the number of actual rounds of communication in the final protocol is $2\log n$ rather than $\log n$.

The setup. Suppose at the start of Round i, the prover has claimed to know two vectors $u^{(i-1)}, \mathbf{g}^{(i-1)} \in \mathbb{G}^{n \cdot 2^{-(i-1)}}$ satisfying the following three equations:

$$\langle u^{(i-1)}, \mathbf{g}^{(i-1)} \rangle = c_1, \tag{15.17}$$

$$\langle u^{(i-1)}, \Gamma^{(i-1)} \rangle = c_2 \tag{15.18}$$

$$\langle \mathbf{g}^{(i-1)}, \Gamma^{(i-1)} \rangle = c_3. \tag{15.19}$$

In Round 1, the following choices ensure that the three equations above capture $\mathcal{P}$'s claim that the protocol as a whole is meant to verify, namely that $\mathcal{P}$ knows $u^{(0)}$ satisfying

$$\langle u^{(0)}, \mathbf{g}^{(0)} \rangle = c_{u^{(0)}}. \tag{15.20}$$

Set $\Gamma^{(0)} = \mathbf{g}^{(0)}$, and set $c_1 = c_2 = c_{u^{(0)}}$. Finally, set $c_3 = \langle \mathbf{g}^{(0)}, \mathbf{g}^{(0)} \rangle$, which is a quantity that can be included in the output of pre-processing, as it is independent of $u^{(0)}$.[190] Under the above settings of $\Gamma^{(0)}$, c_1, c_2, and c_3, the validity of Equations (15.17)-(15.19) for $i = 1$ is equivalent to the validity of Equation (15.20).

Description of Round *i*. The purpose of Round i is to reduce the three equations above to three equations of the same form, but over vectors $u^{(i)}$ and $g^{(i)}$ that are shorter than $u^{(i-1)}, \mathbf{g}^{(i-1)}$ by a factor of 2. This is done as follows (below, we intermingle the description of each step of the protocol with intuitive justification for the step. A standalone protocol description is in Protocol 14).

- The prover begins by sending four quantities $D_{1L}, D_{1R}, D_{2L}, D_{2L} \in \mathbb{G}_t$ claimed to be AFGHO-commitments to the left and right halves of $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ under key $\Gamma^{(i)}$. Conceptually, the point of sending these four commitments is to enable the verifier to homomorphically compute commitments under $\Gamma^{(i)}$ to $u^{(i)}$ and $\mathbf{g}^{(i)}$ (defined later in the protocol).
- The verifier chooses a random $\beta \in \mathbb{F}_p$ and sends it to $\mathcal{P}$. Conceptually, β is used to "take a random linear combination" of the three equations, yielding a single "equivalent" equation. Specifically, observe that if Equations (15.17)-(15.19) hold, then so does the following equation:

  $$\langle u^{(i-1)} + \beta \Gamma^{(i-1)}, \mathbf{g}^{(i-1)} + \beta^{-1} \Gamma^{(i-1)} \rangle = c_1 + \beta^{-1} c_2 + \beta c_3 + \langle \Gamma^{(i-1)}, \Gamma^{(i-1)} \rangle. \tag{15.21}$$

  Meanwhile, it turns out that if the prover can establish that it knows $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ such that Equation (15.21) holds even for just three values of β, then in fact $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ satisfy Equations (15.17)-(15.19). This is because, if any one of Equations (15.17)-(15.19) fail to hold, then Equation (15.21) can only hold for at most 2 values of $\beta \in \mathbb{F}_p$, as the left hand and right hand sides of Equation (15.21) are distinct Laurent polynomials in β, and hence can agree on at most 2 points.

  So, conceptually speaking, the remainder of Round i will be devoted to proving that Equation (15.21) holds for the verifier's random choice of β. As we detail below, the remainder of the round accomplishes this essentially by applying the standard Bulletproofs iteration to Equation (15.21), i.e., of randomly choosing an $\alpha \in \mathbb{F}_p$ and using it to "randomly combine" the left and right halves of $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ respectively to obtain new vectors $u^{(i)}$ and $\mathbf{g}^{(i)}$of half the length.

  To this end, let

  $$w_1 = u^{(i-1)} + \beta \Gamma^{(i-1)} \tag{15.22}$$

[190] More generally, Round i of the protocol will require that the pre-processing also output the quantity $\langle \Gamma^{(i-1)}, \Gamma^{(i-1)} \rangle$.

and

$$w_2 = \mathbf{g}^{(i-1)} + \beta^{-1}\Gamma^{(i-1)}, \tag{15.23}$$

and let w_{1L} and w_{1R} denote the left and right halves of w_1 and similarly for w_{2L} and w_{2R}.

- The prover sends cross-terms v_L and v_R claimed to equal $\langle w_{1L}, w_{2R}\rangle$ and $\langle w_{1R}, w_{2L}\rangle$.
- The verifier chooses a random $\alpha \in \mathbb{F}_p$ and sends it to the prover.
- The prover defines:

$$u^{(i)} = \alpha \cdot w_{1L} + \alpha^{-1} w_{1R} \tag{15.24}$$

$$\mathbf{g}^{(i)} = \alpha^{-1} \cdot w_{2L} + \alpha w_{2R}. \tag{15.25}$$

- The verifier does not know $u^{(i)}$ or $\mathbf{g}^{(i)}$ but can homomorphically compute the following three quantities:
  - $\langle u^{(i)}, \mathbf{g}^{(i)}\rangle$. Indeed, if Equations (15.17)-(15.19) hold and v_L, v_W are prescribed, then a straightforward consequence of Equation (15.21) is that

    $$\langle u^{(i)}, \mathbf{g}^{(i)}\rangle = c_1 + \beta^{-1} c_2 + \beta c_3 + \langle \Gamma^{(i-1)}, \Gamma^{(i-1)}\rangle + \alpha^2 v_L + \alpha^{-2} v_R. \tag{15.26}$$

    Note that the verifier has access to all terms on the right hand side of Equation (15.26) (as mentioned in Footnote 190, $\langle \Gamma^{(i-1)}, \Gamma^{(i-1)}\rangle$ can be computed in pre-processing.)
  - $\langle u^{(i)}, \Gamma^{(i)}\rangle$. Indeed, if D_{1L} and D_{1R} are as prescribed, then Equations (15.22) and (15.24) imply that:

    $$\langle u^{(i)}, \Gamma^{(i)}\rangle = \alpha D_{1L} + \alpha^{-1} D_{1R} + \alpha\beta\Delta_L^{(i)} + \alpha^{-1}\beta\Delta_R^{(i)}. \tag{15.27}$$

    Recall that, here, $\Delta_L^{(i)} = \langle \Gamma_L^{(i-1)}, \Gamma^{(i)}\rangle$ and $\Delta_R^{(i)} = \langle \Gamma_R^{(i-1)}, \Gamma^{(i)}\rangle$ are commitments to the left and right halves of $\Gamma^{(i-1)}$ computed during pre-processing.
  - $\langle \mathbf{g}^{(i)}, \Gamma^{(i)}\rangle$. If D_{2L} and D_{2R} are as prescribed, then Equations (15.23) and (15.25) imply that

    $$\langle \mathbf{g}^{(i)}, \Gamma^{(i)}\rangle = \alpha^{-1} D_{2L} + \alpha D_{2R} + \alpha^{-1}\beta^{-1}\Delta_L^{(i)} + \alpha\beta^{-1}\Delta_R^{(i)}. \tag{15.28}$$

- Let c_1', c_2', and c_3' denote the above three quantities that the verifier homomorphically computes. Round $i+1$ is then devoted to showing that indeed the prover knows $u^{(i)}, \mathbf{g}^{(i)}$ such that the following three equations indeed hold:

$$\langle u^{(i)}, \mathbf{g}^{(i)}\rangle = c_1'$$
$$\langle u^{(i)}, \Gamma^{(i)}\rangle = c_2',$$
$$\langle \mathbf{g}^{(i)}, \Gamma^{(i)}\rangle = c_3'.$$

The above protocol is summarized in Protocol 14.

Sketch of the extraction analysis. The knowledge extractor proceeds similarly to the one for the protocol of Section 15.4.3. First, it constructs a 3-transcript tree for the protocol, of depth $2\log n$ (the 2 comes in because each of the $\log n$ "conceptual rounds" in the protocol description actually consists of 2 communication rounds). For the remainder of this sketch, we use the phrase "round" to refer to a conceptual round.

As usual, the extractor proceeds from the leaves toward the root. At each node of distance $2i$ from the root (capturing the start of round $i+1$ of the protocol), it constructs vectors $u^{(i)}$ and $\mathbf{g}^{(i)}$ that "explain" the prover's messages in all transcripts of the sub-tree rooted at that node.

Suppose by way of induction that the extractor has already succeeded in reconstructing such vectors for all vertices in the tree corresponding to rounds $i+1$ and later. Consider a tree vertex corresponding to round $i+1$ of the protocol, and let α, β denote the random group elements selected by the verifier in round i. Because AFGHO commitments are computationally binding and the extractor is efficient, one can argue that for the vector $u^{(i+1)}$ reconstructed by the extractor for that vertex, there is an (efficiently computable) vector $z = (z_L, z_R)$ such that

$$u^{(i+1)} = \alpha z_L + \alpha^{-1} z_R + \alpha\beta\Gamma_L^{(i-1)} + \alpha^{-1}\beta\Gamma_R^{(i-1)}.$$

This is because Equation (15.27) ensures that the commitment to $u^{(i+1)}$ is a commitment to a vector of this form. Similarly, one can argue that there is an (efficiently computable) vector $t = (t_L, t_R)$ such that the reconstructed vector $\mathbf{g}^{(i+1)}$ is of the form

$$\alpha^{-1} t_L + \alpha t_R + \alpha^{-1}\beta^{-1}\Gamma_L^{(i-1)} + \alpha\beta\Gamma_R^{(i-1)}.$$

Now consider any node of the transcript tree capturing the second message of round i of the protocol. Because the second message of Round i applies the Bulletproofs update procedure to commitments to w_1 and w_2 under commitment key $\Gamma^{(i)}$, the Bulletproofs extractor is able to reconstruct vectors w_1 and w_2 such that Equations (15.24) and (15.25) hold, i.e., w_1 and w_2 "explain" the reconstructed vectors $u^{(i+1)}$ and $\mathbf{g}^{(i+1)}$ for all child vertices of the node. Moreover, the previous paragraph implies that there are efficiently computable vectors $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ such that Equations (15.22) and (15.23) hold, i.e., $w_1 = u^{(i-1)} + \beta\Gamma^{(i-1)}$ and $w_2 = \mathbf{g}^{(i-1)} + \beta^{-1}\Gamma^{(i-1)}$.

All that is left is to argue that $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ satisfy Equations (15.17)-(15.19). That $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ satisfy Equation (15.21) for the three values of β appearing in the 3-transcript tree follows from the following three facts. First, the verifier's commitment computation in Equation (15.26) applies the Bulletproofs update procedure to confirm that the prover knows vectors $w_1 = u^{(i-1)} + \beta\Gamma^{(i-1)}$ and $w_2 = \mathbf{g}^{(i-1)} + \beta^{-1}\Gamma^{(i-1)}$ such that Equation (15.21) holds. Second, as argued above, w_1 and w_2 "explain" the reconstructed vectors $u^{(i+1)}$ and $\mathbf{g}^{(i+1)}$ and hence they are exactly the vectors that the Bulletproofs knowledge extractor would compute if the Bulletproofs update procedure were applied to Equation (15.21). This means that the vectors computed by the extractor satisfy Equation (15.21). Finally, as explained in the protocol description itself, if $u^{(i-1)}$ and $\mathbf{g}^{(i-1)}$ satisfy Equation (15.21) for three or more values of β, then they also satisfy Equations (15.17)-(15.19).

Protocol 14 A transparent argument of knowledge of vectors $u, \mathbf{g} \in \mathbb{G}^n$ such that $\langle u, \mathbf{g} \rangle = c_1$, $\langle u, \Gamma \rangle = c_2$, and $\langle \mathbf{g}, \Gamma \rangle = c_3$, where $\Gamma \in \mathbb{G}^n$ is a public vector summarized via a linear-time pre-processing phase (Section 15.4.3). Here, $\mathbb{G}$ is an additive group and $\langle u, \mathbf{g} \rangle = \sum_{i=1}^{n} e(u_i, g_i)$ denotes the inner-pairing-product between u and $\mathbf{g}$. The protocol consists of $2\log_2 n$ communication rounds, with 6 $\mathbb{G}_t$ elements communicated from prover to verifier per round. Total verifier time is dominated by $O(\log n)$ scalar multiplications in $\mathbb{G}_t$. For simplicity, we present the protocol for a symmetric pairing, though inner-pairing-product commitments are only binding for asymmetric pairings (in particular, under the SXDH assumption, which only holds for asymmetric pairings).

1: Let $\mathbb{G}$ be an additive cyclic group of prime order p with with bilinear map $e \colon \mathbb{G} \times \mathbb{G} \to \mathbb{G}_t$.
2: Input is $c_1 = \langle u, \mathbf{g} \rangle$, $c_2 = \langle u, \Gamma \rangle$, and $c_3 = \langle \mathbf{g}, \Gamma \rangle$, where $u, \mathbf{g}, \Gamma \in \mathbb{G}^n$. Verifier only knows c_1, c_2, c_3 and the following quantities computed during pre-processing: $\langle \Gamma, \Gamma \rangle$, as well as AFGHO commitments $\Delta_L = \langle \Gamma_L, \Gamma' \rangle$ and $\Delta_R = \langle \Gamma_R, \Gamma' \rangle$ to the left and right halves of Γ under public commitment key $\Gamma' \in \mathbb{G}^{n/2}$.
3: If $n = 1$, Prover sends u, $\mathbf{g}$, and Γ, to the verifier and the verifier checks that $c_1 = \langle u, \mathbf{g} \rangle$, $c_2 = \langle u, \Gamma \rangle$, $c_3 = \langle \mathbf{g}, \Gamma \rangle$, $\Delta_L = \langle \Gamma_L, \Gamma' \rangle$ and $\Delta_R = \langle \Gamma_R, \Gamma' \rangle$.
4: Otherwise, $\mathcal{P}$ sends $D_{1L}, D_{1R}, D_{2L}, D_{2L} \in \mathbb{G}_t$ claimed to be AFGHO-commitments to the left and right halves of u and $\mathbf{g}$ under key Γ'.
5: $\mathcal{V}$ chooses a random $\beta \in \mathbb{F}_p$ and sends it to $\mathcal{P}$.
6: $\mathcal{P}$ sets $w_1 = u + \beta\Gamma$ and $w_2 = \mathbf{g} + \beta^{-1}\Gamma$. Let w_{1L} and w_{1R} denote the left and right halves of w_1 and similarly for w_{2L} and w_{2R}.
7: $\mathcal{P}$ sends $v_L, v_R \in \mathbb{G}_t$ claimed to equal $\langle w_{1L}, w_{2R} \rangle$ and $\langle w_{1R}, w_{2L} \rangle$.
8: $\mathcal{V}$ chooses a random $\alpha \in \mathbb{F}_p$ and sends it to $\mathcal{P}$
9: $\mathcal{P}$ sets $u' = \alpha \cdot w_{1L} + \alpha^{-1} \cdot w_{1R}$ and $\mathbf{g}' = \alpha^{-1} \cdot w_{2L} + \alpha \cdot w_{2R}$.
10: $\mathcal{V}$ sets:

$$c_1' = c_1 + \beta^{-1} c_2 + \beta c_3 + \langle \Gamma, \Gamma \rangle + \alpha^2 v_L + \alpha^{-2} v_R.$$
$$c_2' = \alpha D_{1L} + \alpha^{-1} D_{1R} + \alpha\beta\Delta_L + \alpha^{-1}\beta\Delta_R.$$
$$c_3' = \alpha^{-1} D_{2L} + \alpha D_{2R} + \alpha^{-1}\beta^{-1}\Delta_L + \alpha\beta^{-1}\Delta_R.$$

11: $\mathcal{V}$ and $\mathcal{P}$ apply the protocol recursively to prove that $\mathcal{P}$ knows vectors $u', \mathbf{g}' \in \mathbb{G}^{n/2}$ such that $\langle u', \mathbf{g}' \rangle = c_1'$, $\langle u', \Gamma' \rangle = c_2'$, and $\langle \mathbf{g}', \Gamma' \rangle = c_3'$.